

3. 추론 과 LLM 연동

데이터에서 결과가 나오는 메커니즘. TypeQL fun 의 핵심 트릭 + Python ↔ TypeQL Write-back + LLM 질의 흐름.

1. TypeQL — 스키마 / fun / 데이터 3단 구조

스키마 (데이터 카드의 설계도)

```
define
  threshold-set sub entity,
    has config-version, # v1, v2 등 정책 버전
    has th-low,         # 단계별 경계값 (예: 0.20)
    has th-mid-low,
    has th-mid-high,
    has th-high;
```

fun (DB 내부에 저장된 판단 로직)

```
define
  fun s_priority_to_state($sp, $version) -> { string }:
    match
      # 1. 정책 버전($version)에 해당하는 노드를 DB에서 조회
      $t isa threshold-set, has config-version $version;
      $t has th-low $tl, has th-mid-low $tml,
        has th-mid-high $tmh, has th-high $th;

      # 2. 동적으로 조회한 경계값과 점수 비교
      if ($sp < $tl) { fetch { "GeneralManagement" }; }
      else if ($sp < $tml) { fetch { "EnhancedMonitoring" }; }
      else if ($sp < $tmh) { fetch { "ReviewPreWatering" }; }
      else if ($sp < $th) { fetch { "PriorityPreWatering" }; }
      else { fetch { "ImmediatePreWatering" }; }
```

🎯 **핵심 트릭:** 함수 내부에 임계값 숫자를 하드코딩하지 않고, `match` 문으로 DB 내 정책 노드를 동적으로 조회한다.

데이터 (실제 정책 카드)

```
insert
  $v1 isa threshold-set, has config-version "v1",
    has th-low 0.20, has th-mid-low 0.40,
    has th-mid-high 0.60, has th-high 0.80;

  $v2 isa threshold-set, has config-version "v2",
    has th-low 0.15, has th-mid-low 0.30,
    has th-mid-high 0.50, has th-high 0.70;
```

→ 새 정책 v3 를 추가하려면: 코드 0줄, INSERT 한 줄. 함수는 매개변수만 바꾸면 즉시 새 정책으로 동작.

2. Python → TypeQL Write-back 4단계

추론 결과가 어떻게 "데이터" 가 되는가 — 4단계.

```
[Step 1. 조회] Python → TypeQL (match/fetch)
"여수 상암동의 인구·바람 알려줘"
→ population=8000, wind_speed=16.0

[Step 2. 계산] Python (formulas.py)
score = calculate_priority(8000, 16.0)
→ 0.560 (예시)

[Step 3. 쿼리 생성] Python → TypeQL (insert)
query = f"match $s isa segment, has segment-id 'EMD_여수_상암동';
insert $s has s-priority {0.560};"

[Step 4. 기록] DB
(Before) segment — has segment-id "EMD_여수_상암동"
(After)  segment — has segment-id "EMD_여수_상암동"
                    └─ has s-priority 0.560 ← NEW 영구 추가
```

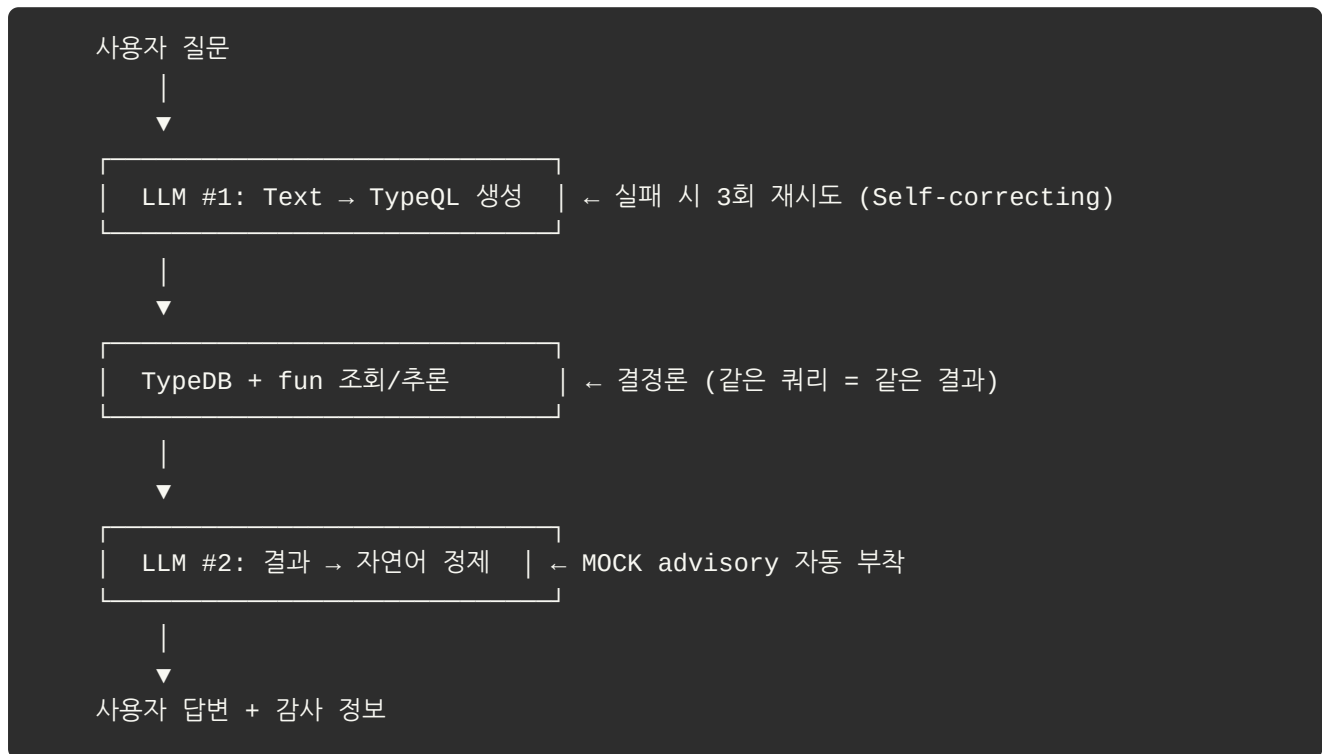
역할 분담

단계	주체	동작
1. 조회	Python → TypeQL	DB 의 기초 사실 읽기
2. 계산	Python	메모리 상에서 가중치 산식
3. 생성	Python → TypeQL	insert 문 동적 조립
4. 기록	TypeQL (DB)	노드에 attribute 로 영구 저장

Python = 두뇌, TypeQL = 손. 두뇌의 생각을 DB 에 도장 찍는 게 TypeQL.

3. 사용자 질의 흐름 (LLM #1 + DB + LLM #2)

자연어 질문이 답변으로 나오는 흐름.



두 LLM 호출의 역할 차이

	LLM #1	LLM #2
입력	사용자의 자연어 질문	DB 조회 결과 (JSON)
출력	TypeQL 쿼리 문자열	자연어 답변
실패 시	3회까지 재시도 → golden fallback	정제 실패 시 raw JSON 노출
역할	사용자 의도 → 그래프 언어	그래프 결과 → 사람 언어

사실 생성은 DB+fun, 표현 변환만 LLM. 환각이 끼어들 자리가 없다.

4. 환각 방지의 두 축

축 1 — MOCK advisory

LLM 이 MOCK 데이터를 인용해 답할 때, 응답에 "이건 추정값입니다" 가 자동으로 따라붙는다. source 의 `availability-status` 가 답변까지 따라가는 구조.

축 2 — 데이터 부재 시 거부

DB 에 없는 데이터를 LLM 이 추측해서 답하지 않는다. 명확히 거부한다.

Q: "1990년 1월 광주 동구 위험도?"
A: "해당 시점의 데이터가 존재하지 않습니다. (실패 처리 - 추측하지 않음)"

→ 사실 생성을 LLM 에서 떼어낸 결과.

5. 응답 JSON = 모든 출력의 단일 원천

추론 결과 JSON 1개가 본 시스템의 세 출력으로 흘러간다.



- V-World GIS: `state` 별 색상 히트맵
- Superset 표: `s_priority` 랭킹표
- 감사 로그: `run_context_id` + `git_commit` 으로 "어느 시점, 어떤 코드로 나온 결과인가" 추적

6. 왜 이 구조인가 — 사실 vs 표현의 분리

전통적인 LLM 시스템: LLM 이 사실도 만들고 표현도 한다 → 환각 발생.

본 시스템: 사실은 그래프와 함수가, 표현은 LLM 이. 두 책임을 분리하면 환각 영역이 차단된다.

책임	담당	특징
사실 생성	TypeDB + fun (Python 산식)	결정론, 재현 가능, 추적 가능
표현 변환	LLM (Gemini → 운영 시 내부 LLM)	자연어로의 변환만

→ 이렇게 책임을 분리한 게 본 시스템의 환각 방지·감사 추적·모듈성 설계의 핵심이다.

 한 줄 요약

"fun = DB 안의 판단 로직 + 정책 노드 동적 조회. Python(두뇌) ↔ TypeQL(손) Write-back. LLM 은 표현만 담당해 환각 차단."